

State Management Activities using Redux Toolkit and Zustand

1. Project Initialization

```
npm create vite@latest react-state-demo -- --template react-ts
cd react-state-demo
npm install
```

2. Approach A: Zustand Implementation

Zustand is minimal and stores state in a single, vanilla JavaScript object. TypeScript types are inferred easily, and no React context provider wrapper is necessary.

Step 1: Install Zustand

```
npm install zustand
```

Step 2: Define Types & Create the Store

Create a file named `src/store/zustandCartStore.ts`. This store manages an array of cart items, handles quantity updates, adds new products safely, and exposes calculated values.

```
import { create } from 'zustand';

export interface CartItem {
  id: string;
  name: string;
  price: number;
  quantity: number;
}

interface CartState {
  items: CartItem[];
  addToCart: (product: Omit<CartItem, 'quantity'>) => void;
  removeFromCart: (id: string) => void;
  updateQuantity: (id: string, quantity: number) => void;
  clearCart: () => void;
  getTotalPrice: () => number;
}

export const useZustandCart = create<CartState>((set, get) => ({
  items: [],

  addToCart: (product) => set((state) => {
    const existingItem = state.items.find((item) => item.id === product.id);
    if (existingItem) {
      return {
        items: state.items.map((item) =>
          item.id === product.id ? { ...item, quantity: item.quantity + 1 } : item
        ),
      },
    }
  })
}));
```

```

    };
  }
  return { items: [...state.items, { ...product, quantity: 1 }] };
}),

removeFromCart: (id) => set((state) => ({
  items: state.items.filter((item) => item.id !== id),
})),

updateQuantity: (id, quantity) => set((state) => ({
  items: state.items.map((item) =>
    item.id === id ? { ...item, quantity: Math.max(1, quantity) } : item
  ),
})),

clearCart: () => set({ items: [] }),

// Computed state using get()
getTotalPrice: () => {
  return get().items.reduce((total, item) => total + item.price * item.quantity, 0);
},
}));

```

Step 3: Consume Zustand in Components

Create src/components/ZustandCart.tsx to read state and trigger actions.

```

import { useZustandCart } from '../store/zustandCartStore';

export const ZustandCart = () => {
  const { items, addToCart, removeFromCart, updateQuantity, getTotalPrice, clearCart } =
    useZustandCart();

  const demoProduct = { id: 'p1', name: 'Wireless Headphones', price: 99.99 };

  return (
    <div style={{ padding: '20px', border: '1px solid #ccc', borderRadius: '8px' }}>
      <h2>Zustand Shopping Cart</h2>
      <button onClick={() => addToCart(demoProduct)}>Add Wireless Headphones ($99.99)</button>

      {items.length === 0 ? (
        <p>Your cart is empty.</p>
      ) : (
        <div>
          <ul>
            {items.map((item) => (
              <li key={item.id} style={{ margin: '10px 0' }}>

```

```

        <strong>{item.name}</strong> - ${item.price} x {item.quantity}
        <button style={{ marginLeft: '10px' }} onClick={() => updateQuantity(item.id, item.quantity - 1)}>-
</button>
        <button style={{ marginLeft: '5px' }} onClick={() => updateQuantity(item.id, item.quantity +
1)}>+</button>
        <button style={{ marginLeft: '15px', color: 'red' }} onClick={() =>
removeFromCart(item.id)}>Remove</button>
    </li>
  )}
</ul>
<h3>Total: ${getTotalPrice().toFixed(2)}</h3>
<button onClick={clearCart}>Clear Cart</button>
</div>
)}
</div>
);
};

```

3. Approach B: Redux Toolkit (RTK) Implementation

Redux Toolkit uses a strictly structured data flow consisting of Slices, a global Store, and special React bindings. It uses Immer internally, allowing you to write code that looks like it mutates state directly.

Step 1: Install Redux Dependencies

```
npm install @reduxjs/toolkit react-redux
```

Step 2: Create the Cart Slice

Create a file named src/store/cartSlice.ts.

```

import { createSlice, PayloadAction } from '@reduxjs/toolkit';

export interface CartItem {
  id: string;
  name: string;
  price: number;
  quantity: number;
}

interface CartState {
  items: CartItem[];
}

const initialState: CartState = {
  items: [],
};

const cartSlice = createSlice({

```

```

name: 'cart',
initialState,
reducers: {
  addToCart: (state, action: PayloadAction<Omit<CartItem, 'quantity'>>) => {
    const existingItem = state.items.find((item) => item.id === action.payload.id);
    if (existingItem) {
      existingItem.quantity += 1;
    } else {
      state.items.push({ ...action.payload, quantity: 1 });
    }
  },
  removeFromCart: (state, action: PayloadAction<string>) => {
    state.items = state.items.filter((item) => item.id !== action.payload);
  },
  updateQuantity: (state, action: PayloadAction<{ id: string; quantity: number }>) => {
    const item = state.items.find((i) => i.id === action.payload.id);
    if (item) {
      item.quantity = Math.max(1, action.payload.quantity);
    }
  },
  clearCart: (state) => {
    state.items = [];
  },
},
});

export const { addToCart, removeFromCart, updateQuantity, clearCart } = cartSlice.actions;
export default cartSlice.reducer;

```

Step 3: Configure the Store & Typed Hooks

Create a file named `src/store/reduxStore.ts`. We must export explicitly typed hooks so components know exactly what the global state looks like.

```

import { configureStore } from '@reduxjs/toolkit';
import cartReducer from './cartSlice';
import { TypedUseSelectorHook, useDispatch, useSelector } from 'react-redux';

export const reduxStore = configureStore({
  reducer: {
    cart: cartReducer,
  },
});

// Calculate and export types derived directly from the store setup
export type RootState = ReturnType<typeof reduxStore.getState>;
export type AppDispatch = typeof reduxStore.dispatch;

```

```
// Custom hooks to bypass manually re-typing state every time
export const useAppDispatch = () => useDispatch<AppDispatch>();
export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector;

// Selector to calculate derived data
export const selectCartTotal = (state: RootState) =>
  state.cart.items.reduce((total, item) => total + item.price * item.quantity, 0);
```

Step 4: Consume Redux inside Components

Create src/components/ReduxCart.tsx.

```
import { useAppDispatch, useAppSelector, selectCartTotal } from '../store/reduxStore';
import { addToCart, removeFromCart, updateQuantity, clearCart } from '../store/cartSlice';

export const ReduxCart = () => {
  const items = useAppSelector((state) => state.cart.items);
  const total = useAppSelector(selectCartTotal);
  const dispatch = useAppDispatch();

  const demoProduct = { id: 'p2', name: 'Mechanical Keyboard', price: 149.99 };

  return (
    <div style={{ padding: '20px', border: '1px solid #ccc', borderRadius: '8px', marginTop: '20px' }}>
      <h2>Redux Toolkit Shopping Cart</h2>
      <button onClick={() => dispatch(addToCart(demoProduct))}>Add Mechanical Keyboard
        ($149.99)</button>

      {items.length === 0 ? (
        <p>Your cart is empty.</p>
      ) : (
        <div>
          <ul>
            {items.map((item) => (
              <li key={item.id} style={{ margin: '10px 0' }}>
                <strong>{item.name}</strong> - ${item.price} x {item.quantity}
                <button style={{ marginLeft: '10px' }} onClick={() => dispatch(updateQuantity({ id: item.id,
                  quantity: item.quantity - 1 })))>-</button>
                <button style={{ marginLeft: '5px' }} onClick={() => dispatch(updateQuantity({ id: item.id, quantity:
                  item.quantity + 1 })))>+</button>
                <button style={{ marginLeft: '15px', color: 'red' }} onClick={() =>
                  dispatch(removeFromCart(item.id))}>Remove</button>
              </li>
            ))}
          </ul>
          <h3>Total: ${total.toFixed(2)}</h3>
        </div>
      )}
    </div>
```

```

    <button onClick={() => dispatch(clearCart())}>Clear Cart</button>
  </div>
  })
</div>
);
};

```

4. Hooking Components up to the Main App

To render both components and see them working in isolation, update your main configuration in `src/App.tsx`. Redux requires its explicit `<Provider>` wrapper to distribute its state down the tree, while Zustand remains un-wrapped.

```

import { Provider } from 'react-redux';
import { reduxStore } from './store/reduxStore';
import { ZustandCart } from './components/ZustandCart';
import { ReduxCart } from './components/ReduxCart';

function App() {
  return (
    <div style={{ maxWidth: '600px', margin: '0 auto', fontFamily: 'sans-serif' }}>
      <h1>State Management Comparison</h1>

      {/* Zustand does not need a provider layer */}
      <ZustandCart />

      {/* Redux requires context provision */}
      <Provider store={reduxStore}>
        <ReduxCart />
      </Provider>
    </div>
  );
}

export default App;

```